CMU 15-213 计算机系统导论 - 2017秋季第一课

课程介绍

大家下午好，欢迎来到2017年秋季学期的15-213课程，以及18-213，可能还有一些15-513的学生。这门课程有三个不同的编号，研究生版本（513）和本科生版本（213）之间有些许差异，但大部分情况下我们将其作为一门统一的大型课程来运营——实际上是校园里最大的课程之一。

我叫Randy Bryant，你们可能在购买的教材上看到过我的名字。我是最早教授这门课的教师之一，虽然我不确定具体教了多少次，但次数很多。我仍然非常喜欢这门课，学生们也从中收获良多，这对我和我的合著者Dave O'Hallaron来说意义重大。

今天我还有两位共同授课的教师：Phil Gibbons也会像我一样授课，Brian Railing名义上负责这门课程的研究生部分，但我们在许多方面都通力合作。

课程标语

当我为这门课准备讲义时——大约在你们出生的时候——我输入了这个数字15213很多次。今天我突然想到：这正好是匹兹堡的邮政编码！所以我们的标语是"**我们给CMU一个邮编**"。18213不是有效的区号，15513在堪萨斯的某个地方，所以这个梗对匹兹堡最适用。

课程目标

为什么要学这门课?

这门课叫"计算机系统导论"，名字有点平淡。但这门课的理念是：你们大多数人以前接触计算机的方式比较抽象，可能用Python或其他高级语言编程，与机器真正发生的事情——比特如何设置、字节如何存储、数据包如何在网络中传输——相距甚远。

这门课的目的是让你开始理解计算机内部真正发生了什么。

即使这是你一生中唯一的一门系统课程，你也能学到一些非常有用的知识，可以应用到工作、进一步的学习或其他任何endeavors中。你会更理解那些奇怪的特性，知道如何深入挖掘和修补代码。

为后续课程打基础

对于那些继续学习其他课程的同学，特别是计算机科学和电子工程的课程，如网络、编译器、操作系统、分布式系统、计算机图形学、嵌入式系统、计算机体系结构等，**这门课是你的入门点**。多年来，所有这些课程都已演变为假设你已经掌握了本课程的内容。

事实上，这门课的起源就是为了帮助学生为操作系统课程做准备。

为什么抽象不总是有效

在理想世界中，我们不需要了解很多这些东西——一切都应该正常工作。但现实是事情从来不会完全按预期运行。如果你想：

- 修复出问题的东西
- 确保系统安全
- 添加原本不可用的功能

你必须更深入地理解事物的工作原理。

计算机科学和所有工程学科都基于一套有原则的抽象。比如在物理学中，牛顿力学是一个抽象，它在大多数情况下有效，直到你接近光速时就不太管用了。

这个原则在科学和工程领域都很深刻：**存在某种抽象层，90%的时间都能工作，用这种抽象思考非常有用，但抽象并不总是成立**。这在计算机科学和计算机工程中反复出现。

这门课的部分目的是探索程序员通常使用的抽象与硬件和系统软件层面真正发生的事情之间的边界。

核心主题示例

1. 整数不是数学意义上的整数

示例：平方数应该非负

在代数中我学到，如果对一个数求平方，结果应该总是大于或等于零（无论是实数、有理数还是整数）。让我们测试这个假设：

```c
// sq.c - 简单的平方函数
int square(int x) {
    return x * x;
}
```

运行结果：

- `square(40000)` = 1,600,000,000 ✓ 正确
- `square(50000)` = -1,794,967,296 ✗ 负数！

为什么？ 在这台Linux机器上使用32位字长，所有整数都用32位表示。50,000的平方太大，无法用这种表示法容纳，所以发生了**溢出**，导致位模式发生了很大变化。

想象一下，如果你在控制火箭，需要50,000的平方，但系统给你返回一个负数——这可能导致灾难！实际上确实发生过类似的火箭失控事件。

Python的不同之处： 在Python中，同样的操作会正常工作，因为Python使用可变长度表示，会根据需要添加更多位来表示大数。理解不同语言能做什么是系统编程的重要部分。

2. 浮点数：范围更大，精度受限

示例：结合律失效

数学中的结合律：(a + b) + c = a + (b + c)

但在浮点运算中：

```python
# 使用GDB调试器演示
(10^20 - 10^20) + 3.14 = 3.14 ✓
10^20 + (-10^20 + 3.14) = 0   ✗
```

为什么？ 当你将3.14加到-10^20时，用科学计数法表示时，10^20远远大于3.14，所以和仍然是-10^20（精度丢失）。然后加上10^20得到零。

浮点数可以处理更大的范围，但代价是精度。如果不理解这一点，在进行数值计算时可能会遇到大麻烦。

几年前，计算机图形学课程将这门课设为先修课，因为他们希望学生理解浮点运算。

3. 汇编语言/机器级编程

虽然现在很少有人直接用汇编语言编写大量代码，但我们发现理解机器级代码对于：

- 理解编译器如何将C代码转换为机器码
- 调试运行缓慢的程序
- 发现奇怪的bug
- 识别攻击者可能利用的安全漏洞

都非常重要。我们会花相当多的时间从高到低、从低到高双向查看程序。

4. 内存层次结构

现代计算机给你一种可以访问大量内存的抽象，但实际上：

- 物理内存是有限的
- 存在内存层次结构：从处理器芯片内的缓存，到外部芯片，甚至到磁盘驱动器
- 操作系统和硬件协同工作，让你感觉可以访问大量内存
- 但性能特征因内存访问模式而异

这个特性变得越来越重要。现在可以轻松获得32或64GB的RAM，但其访问速度远慢于处理器的处理速度。

示例：越界访问

```c
typedef struct {
    int a[2];
    double d;
} struct_t;

void set_val(struct_t *ptr, int i, int val) {
    ptr->a[i] = val;
}

double fun(int i) {
    struct_t s;
    s.d = 3.14;
    set_val(&s, i, 0);
    return s.d;
}
```

在C或C++中没有边界检查！你可以引用`a[39]`，它会存储到完全不相关的内存位置。

运行结果：

- `fun(0)` 或 `fun(1)` → 返回 3.14 ✓
- `fun(2)` → 返回奇怪的数字（覆盖了double的一部分）
- `fun(3)` → 返回 2.0（覆盖了double的高位）
- `fun(4)` → 系统崩溃

这是C/C++中最痛苦的bug类别之一——程序可能运行数分钟、数小时、数天或数周，然后在某个地方引用被破坏的内存位置时崩溃。但你能找出bug是在哪里注入的吗？几乎不可能。

5. 程序性能优化

除了大O符号和渐近复杂度（比如快速排序O(n log n)比冒泡排序O(n^2)好），还有很多其他因素。

有时在工程世界中，你需要的只是让代码快两倍——比如视频编码器需要达到30帧/秒而你只有15帧/秒。这不是渐近复杂度问题，而是常数因子问题。

如果你足够了解程序执行，通常系统中有很多slack可以克服。

示例：内存访问模式

两个函数执行完全相同的操作——将源数组复制到目标数组（2048×2048）：

```c
```

```
// 按行复制
void copy_ij(int src[2048][2048], int dst[2048][2048]) {
    for (int i = 0; i < 2048; i++)
        for (int j = 0; j < 2048; j++)
            dst[i][j] = src[i][j];
}

// 按列复制
void copy_ji(int src[2048][2048], int dst[2048][2048]) {
    for (int j = 0; j < 2048; j++)
        for (int i = 0; i < 2048; i++)
            dst[i][j] = src[i][j];
}
```

性能差异：20倍！ 按行访问快得多，因为它与内存层次结构预期的访问模式一致。

我们用"内存山"（Memory Mountain）来可视化这种性能特征——这是由一位来自瑞士的研究生开发的可视化方法。

6. 网络编程基础

除了编写计算数字或访问内存的程序，计算机系统还有很多其他有趣的部分，特别是通过网络将计算机连接起来。

我们不会深入探讨（需要专门的网络或分布式系统课程），但会给你一个起点，让你更好地理解：

- 使用无线访问时计算机发生了什么
- 通过浏览器获取网页时发生了什么

课程特色：程序员视角

与大多数系统课程不同，这门课采用**程序员中心**的视角，而不是**系统构建者**的视角。

- **操作系统课程**：编写操作系统核心代码
- **编译器课程**：编写编译器
- **计算机体系结构课程**：设计微处理器

但对于系统入门，我们采用不同的方法：**作为C程序员，如何编写能够有效运用和交互系统不同部分的代码?**

这种方法：

1. 让你在深入系统设计之前先理解系统的用途和价值
2. 即使你不再学习其他系统课程，也能学到有用的知识

实际上，设计缓存内存、设计微处理器、实现操作系统的人只占所有使用这些技术的人的一小部分。我们在培养你成为更广泛的程序员群体。

后续课程

这门课是许多课程的先修课程，包括：

- 操作系统
- 编译器
- 网络
- 分布式系统
- 计算机图形学
- 嵌入式系统
- 计算机体系结构

所有这些高级课程都期望你已经掌握了本课程的内容。这门课是CMU计算机科学和电子工程专业的核心课程，也被其他很多专业的学生选修。

学术诚信

（Brian介绍教育研究部分）

我们每学期都会进行一些教育研究，试图更好地理解如何教授计算机系统。这包括：

1. 持续修订和改进课堂讨论（recitations）
2. 课堂小测验（不计入成绩）

你不需要做任何特别的事情来参与。如果你不想包含你的数据，可以发邮件给我或Chad Hershock。所有数据都会匿名化，直到1月份才会查看，不会影响你的实际成绩。

学术诚信的重要性

学术诚信包括各种形式的作弊和抄袭。在计算机科学中这尤其是个问题，因为有很多工具和技术让获取不当信息变得容易和诱人。

我们的特殊责任： 我们希望确保你真正掌握了材料，以便为后续课程做好准备。如果你通过不正当手段完成作业，很可能没有真正学到材料。

基本规则

所有作业（8个实验）必须独立完成：

- ✓ 可以使用教科书和课程网页的信息
- ✓ 可以与助教或教师交流

- ✗ 不应该与其他学生有任何重要的合作
- ✗ 不要从网上获取未经授权的材料

这本教材在全球320所学校使用，网上有很多相关材料。虽然容易找到，但使用这些材料是不允许的。

长期义务：你有义务现在和将来都不向其他学生披露与实验相关的信息。

重要警告：

- 大学的学术诚信政策没有时效限制
- 即使你已经通过课程，如果发现过去有违规行为，我们也可以改变你的成绩，甚至撤销学位
- 我们确实这样做过

GitHub特别问题：很多学生使用GitHub进行版本控制，但要小心：

- 私有账户在一段时间后可能变成公开
- 这会无意中向未来的学生泄露信息

什么不算作弊?

✓ **可以做的**：

- 帮助其他学生解决编译问题、调试器问题等系统级问题
- 讨论非常高层次的设计问题（不是详细代码！）
- 使用书籍网页和课程网页的代码

✗ **高层次设计 ≠ 伪代码**

几年前的案例："我们没作弊，我只是读出我的代码，我朋友在打字。"或者"我们在白板上用伪代码讨论，然后各自编码。"

伪代码就是代码！ 如果详细到需要写下来才能向别人解释，那就不是高层次了。

高层次意思是一般性的："我发现按行访问数组比按列访问更好。"

检测作弊

我们很擅长发现：

- 我在使用网络搜索方面可能和你一样好
- 我们有每学期每个提交的历史记录副本
- 我们运行"作弊检查器"程序（如MOSS）
    - 创建所有历史作业的大池
    - 分析和比较，不会被注释、变量名、缩进改变所愚弄

- 找出匹配项

有些匹配是偶然的，但其他的看起来非常可疑，我们会仔细查看。

后果

最低处罚： 该作业-100%（比不交更糟）

典型处罚： 课程不及格，记录在案（标记为AIV）

大学规定：

- 一次AIV：希望你吸取教训
- 两次AIV：必须在教师和学生组成的委员会前解释，可能被开除

实例：

- 两年前：20名学生（约占5%）因作弊不及格，至少一人后来被开除
- 几年前：发现某些代码在多个学生的作业中出现，追溯几年，对11名已完成课程甚至已毕业的学生进行了重新评分和处罚

个人感受： 我不喜欢做这些事，也不喜欢谈论它。这浪费我的时间，情感上也很困难。对学生来说更加困难。

你们所有人都无意作弊，但当截止日期临近，作业还没完成，你知道网上有现成的代码——这时很容易越界。但后果非常可怕。

关于误判： 有很多误报，有时学生有合理解释。但我们作为教师不需要你的认罪就能施加后果。你有权申诉。

案例场景

❌ **搜索"15-213 bomb lab"** → 能找到YouTube视频，我看过这些视频

❌ **从GitHub获取文件并提交** → 我们能找到并发现这些

✓ **查找库函数的参考材料** → 完全可以

✓ **与助教讨论** → 我们有很多办公时间

典型场景

场景1： 截止日期临近，在GitHub上找到某个学生的代码，复制并稍作修改提交 → 我们会发现并约谈

场景2：角色分析

- Alice在做实验，不工作；Bob已完成，坐在旁边
- Charlie起身休息，Bob打印自己的代码放在Charlie的椅子上

    - **谁作弊了？** Bob（行为可疑），Charlie还不知情
- Charlie回来，看到代码，复制了一个函数
    - **谁作弊了？** Charlie明显作弊
- Bob和Alice一起检查她的代码
    - **谁作弊了？** 两人都作弊（一起工作）
- Charlie看Bob留下的屏幕
    - **谁作弊了？** Charlie作弊；Bob不应该留下未锁定的屏幕，但有一定的隐私期望

✓ **Alice问Bob如何在GDB中设置断点** → 可以，系统级帮助

✓ **Charlie向助教寻求帮助** → 可以

Git版本控制

从去年春季开始，我们强烈建议使用ECE的Git服务器而不是GitHub：

为什么使用Git?

- 恢复到之前的状态（如果改动是坏主意）
- 创建分支
- 不用担心计算机崩溃丢失文件
- 记录你的更改

为什么不用GitHub?

- Git服务器可以作为你工作的**记录**
- 如果我们质疑你的代码，你可以展示一致的工作历史
- 已有学生通过Git日志证明了自己的清白
- 也有学生因为Git记录显示代码突然大量改变（可能导入了新代码）而被发现

Git既对你有利（更好的编程实践），也是一种记录，证明代码确实是你自己完成的。

课程后勤

教学团队

- 三位教师（Randy Bryant, Phil Gibbons, Brian Railing）
- 20位助教

课程结构

- 讲座和课堂讨论（recitations）
- **Canvas平台**： 课堂小测验
    - 不是传统意义上的测验
    - 几个多选题
    - 帮助教师实时了解班级理解程度
    - 不正式计分，但积极参与可能在边界情况下影响成绩
    - 请带笔记本电脑或智能手机

教材

必需:

- 《深入理解计算机系统》第3版（不要买其他版本或国际版）
- 中文版也可以（我们认识译者）

强烈推荐:

- 《C程序设计语言》（K&R）
- 虽然有点过时，但仍然是最好的C语言书
- 不仅讲语言本身，还展示了编程风格和思维方式

作业和考试

8个实验： 课程的主要内容和学习方式

2次考试： 都是在线考试，无纸质测试

资源

- **课程网页**： 大量信息
- **Piazza**： 提问和公告
    - 默认情况下，你的帖子是私密的（只有教师和助教能看到）
    - 如果问题有普遍意义，我们可能将私密帖子转为公开
    - **不要在Piazza发布代码**
- **Autolab**： 提交作业
    - 如果需要代码帮助：提交到Autolab，然后告诉我们
    - 我们可以从Autolab获取你的代码查看

- **办公时间**：除周六外每天下午5:30-7:30在Wean Hall机群（下周开始）
- **Walk-in辅导**：信息将在线发布
- **预约办公时间**：三位教师都有固定办公时间，也可预约

成绩申诉

如果觉得评分不公：

- 教学大纲中有详细程序
- **有时间限制**（通常一周左右）
- 我们理解会犯评分错误，愿意听取申诉
- **但不要等到学期末才来说**Lab 1、Lab 3和期中考试评分有问题——太晚了

自动评分

问：类似122，有自动评分器吗?

答：我们是自动评分之王！

- 我宁愿花3小时写自动评分器，也不愿花1小时批改卷子
- 几乎整个分数都是自动评分
- 只有小部分手动评分（如代码风格分）
- **好处**：快速反馈，有机会修复问题
- 可以多次提交，没有惩罚

重要：确保最后提交的是你想要评分的版本——我们只评最后一次提交

Shark机器

- 为本课程保留的特殊机器
- 大部分工作可以在任何Linux机器上完成
- 但自动评分在特定的Linux服务器上运行
- **有疑问时，使用Shark机器做作业**

宽限政策

宽限日（Grace Days）

- 总共5个宽限日
- 不同作业允许不同数量（0-2天）
- 用于处理日程冲突、工作面试、家庭事务等

- **自动分配，无法控制使用在哪个作业上**

延期提交

- 用完宽限日后，部分作业允许额外延期3天
- 有递增的延迟扣分

特殊情况

- 严重个人问题影响所有课程时
- 联系课程顾问
- 顾问会联系我们制定计划

重要建议：

- 不要过早用完宽限日
- 尽量为最后几个作业保留（这些作业最重要）
- 保持进度——一旦落后很难赶上
- 课程有很强的动量，落后会越来越难追上

课堂政策

- 可以带笔记本电脑，但要做有成效的工作（不是Facebook）

成绩计算

- 50%考试，50%实验
- 不同实验有不同分值（见作业页面）
- **不调分**：标准评分等级
- 常有很多A和B
- 如果你掌握了材料，我们很乐意给好成绩
- 边界情况可能向上调整

Lab 0 - 新实验！

今天可用！

- C编程小练习
- 如果学过122或有C编程经验，应该很简单
- **目的**： 帮助评估和加强C编程技能

- **问题**：一些学生生疏了，一个月后（写更复杂程序时）才发现问题

要求：

- 只占2分
- 应该只需1小时左右
- **截止日期**：下周四

如果昨天中午前注册了课程，应该已有Autolab账户。

其他实验

我们一直在调整和改进实验。使用**Project Zone**系统发放作业说明书：

- 逐步揭示内容
- 回答已看到内容的问题
- 到最后获得一个Pass Key（数字和字母序列）
- 包含在提交中才能提交到Autolab

为什么这样做？ 很多Piazza问题的答案就是"读说明书"。这鼓励你在开始作业前仔细阅读说明书。

Boot Camp - 周一

时间：劳工节（Labor Day），晚上7点 **地点**：Rashid Auditorium
内容：助教讲解Linux和Git

谁应该参加?

- 不熟悉Linux的同学
- 不熟悉命令行工具的同学
- 不熟悉Git的同学

不是强制的，但如果你在这些方面有些薄弱，强烈推荐参加。

总结

期待与大家一起学习！这将是一段充满挑战但收获巨大的旅程。记住：

- 诚实完成作业
- 充分利用资源（助教、办公时间、Piazza）
- 使用Git进行版本控制

- 不要拖延——保持进度
- 遇到问题及时寻求帮助

欢迎来到15-213！